

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study ¹
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 15, 2026

1 Objective

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

2 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (loaded via <code>sklearn.datasets.load_breast_cancer</code> , an identical copy of the canonical UCI WDBC data)
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard-error, and “worst” value of 10 base measurements per cell nucleus)
Number of Classes	2 (Malignant, Benign)
Missing Values	0 (verified programmatically)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code>

Class balance: 357 benign samples (62.7%) and 212 malignant samples (37.3%) – a moderate imbalance. **Label encoding note:** scikit-learn’s built-in loader encodes `target = 0` as *malignant* and `target = 1` as *benign*; this (reversed from the 0=negative/1=positive convention some readers expect) is used consistently throughout this report and all confusion matrices below.

3 Preprocessing Steps

- **Class label encoding:** The dataset’s numeric `target` column (0 = malignant, 1 = benign) was mapped to a readable `diagnosis` column for EDA and reporting purposes; the numeric encoding was used directly for model training.

- **Missing value handling:** None required; the dataset was verified to contain zero missing cells.
- **Feature scaling:** Not required. Both Decision Tree and Random Forest are threshold-split-based models and are invariant to monotonic feature scaling, so no `StandardScaler` step was applied (unlike the distance-/margin-based models in Experiments 2 and 4).
- **Train-test split:** `train_test_split` with `test_size=0.2`, `stratify=y`, `random_state=42`.

4 Exploratory Data Analysis

The reusable function `generate_eda_summary()` from `ml_lab_utils.py` (Experiment 1, unmodified) was used to produce the consolidated 12-panel EDA figure below.

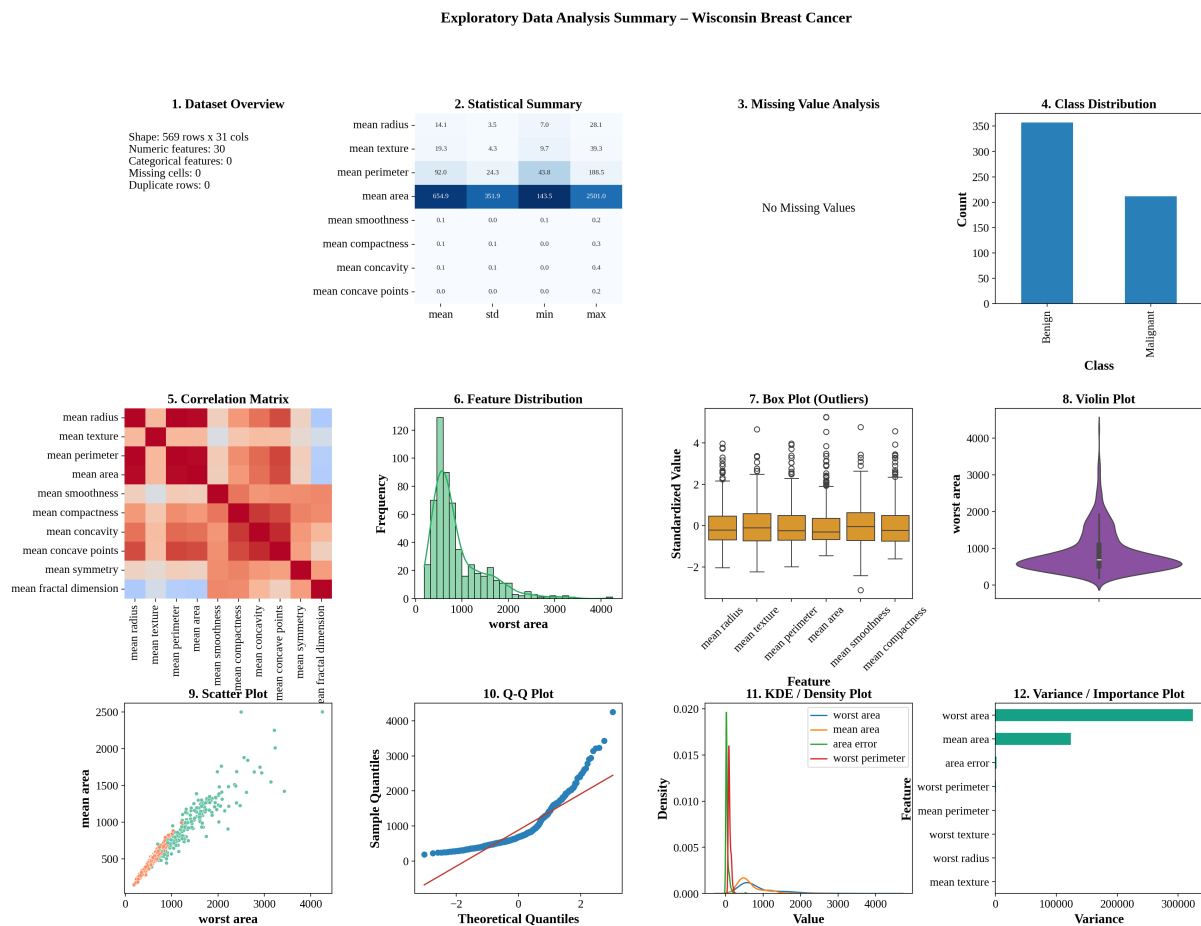


Figure 1: Consolidated 12-panel EDA summary for the Wisconsin Breast Cancer dataset.

Inference: The dataset has no missing values and a moderate class imbalance (62.7% benign, 37.3% malignant). `worst area` and `mean area` are the two highest-variance features (Panels 6, 9, 10, 12) and are, unsurprisingly, very strongly correlated with each other (Panel 9) since both derive from cell/nucleus size. The box plot (Panel 7) shows several high-value outliers among the size-related features, and the Q-Q plot (Panel 10)

confirms **worst area** is right-skewed rather than normally distributed – consistent with a small subset of unusually large, aggressive tumors in the dataset.

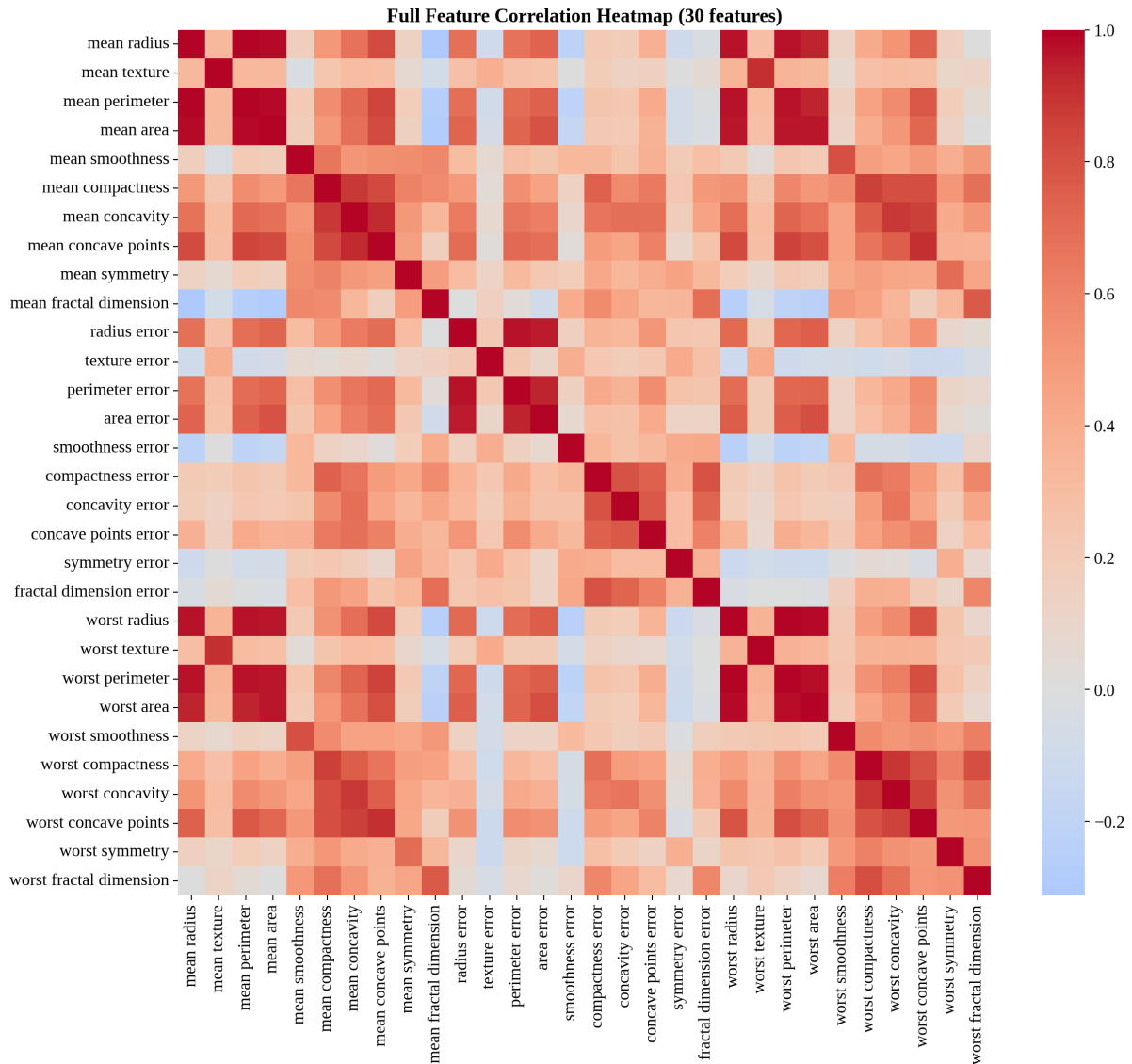


Figure 2: Full 30-feature correlation heatmap, required per Section 2 of the lab manual (Feature correlation analysis).

Inference: The heatmap reveals strong block-diagonal correlation structure: the **mean**, **error**, and **worst** versions of **radius**, **perimeter**, and **area** are all very highly correlated with each other (dark red blocks), as expected since perimeter and area are geometrically derived from radius. Similarly, **concavity**, **concave points**, and **compactness** form a second correlated cluster. This multicollinearity means many of the 30 raw features are redundant, which explains why both models below achieve strong performance using only a handful of the most informative features (Section 10, Feature Importance).

5 Implementation Details

Decision Tree recursively partitions the feature space by choosing, at each node, the feature and threshold that most reduces class impurity (measured by **Gini index** or **entropy**). Deeper trees can fit the training data almost perfectly but are prone to high variance and overfitting, since individual splits become sensitive to noise in small leaf subsets.

Random Forest is an ensemble of many decision trees, each trained on a bootstrap-resampled subset of the training data (**bagging**) and restricted to a random subset of features at each split (**random feature selection**). Final predictions are made by **majority vote** across all trees. Averaging over many de-correlated trees reduces the variance of the ensemble relative to any single tree, generally improving generalization at the cost of reduced interpretability and increased training/prediction cost.

Listing 1: Baseline Decision Tree via the reusable classification function, from `ml_lab_utils.py` (Experiment 1).

```

1 baseline_models = {"Decision Tree (Baseline)":
    DecisionTreeClassifier(random_state=42)}
2 baseline_results_df, baseline_fitted =
    train_evaluate_classification(
3     baseline_models, X_train, X_test, y_train, y_test, scale=
        False)

```

Listing 2: Hyperparameter search spaces and 5-fold GridSearchCV, from `experiment5_dt_rf_breast_cancer.py`.

```

1 dt_param_grid = {
2     "criterion": ["gini", "entropy"],
3     "max_depth": [3, 5, 7, 10, None],
4     "min_samples_split": [2, 5, 10],
5     "min_samples_leaf": [1, 2, 4],
6 }
7 rf_param_grid = {
8     "n_estimators": [50, 100, 200],
9     "max_depth": [5, 10, None],
10    "max_features": ["sqrt", "log2"],
11    "bootstrap": [True, False],
12 }
13 dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
    dt_param_grid,
14                        cv=StratifiedKFold(5, shuffle=True,
15                        random_state=42),
16                        scoring="accuracy")

```

6 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 24.04 LTS
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

7 Hyperparameter Tuning Results

Table 1: Decision Tree Hyperparameter Evaluation using 5-Fold Cross-Validation

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
gini	3	93.19	0.9455
gini	5	93.85	0.9506
gini	7	93.85	0.9506
gini	10	93.85	0.9506
gini	None	93.85	0.9506
entropy	3	93.41	0.9480
entropy	5	92.97	0.9439
entropy	7	92.97	0.9435
entropy	10	92.97	0.9435
entropy	None	92.97	0.9435

Overall best Decision Tree (GridSearchCV, full 4-parameter search): criterion=gini, max_depth=5, min_samples_leaf=4, min_samples_split=2, CV accuracy = 93.85%. Note that accuracy plateaus identically from max_depth=5 onward within the gini rows above because the `min_samples_leaf=4` constraint (selected jointly by the full grid search) naturally stops the tree from growing much beyond depth 5 regardless of the nominal `max_depth` ceiling.

Table 2: Random Forest Hyperparameter Evaluation using 5-Fold Cross-Validation

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1
50	5	sqrt	96.04	0.9687
50	10	log2	96.26	0.9702
50	None	log2	96.26	0.9702
100	5	sqrt	96.04	0.9685
100	10	log2	96.48	0.9718
100	None	log2	96.48	0.9718
200	5	log2	95.82	0.9666
200	10	sqrt	96.26	0.9700
200	None	sqrt	96.26	0.9700

Overall best Random Forest (GridSearchCV, full 4-parameter search):
bootstrap=True, max_depth=10, max_features=log2, n_estimators=100, CV accuracy = 96.48%.

Hyperparameter Tuning Results Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Decision Tree	GridSearchCV (5-fold)	gini, depth=5, leaf=4, split=2	0.9385
Random Forest	GridSearchCV (5-fold)	bootstrap=T, depth=10, log2, n=100	0.9648

8 Decision Tree Performance

Metric	Value
Accuracy	0.9035
Precision	0.9061
Recall	0.9035
F1 Score	0.9041
ROC-AUC	0.9358
Training Time (s)	0.0059

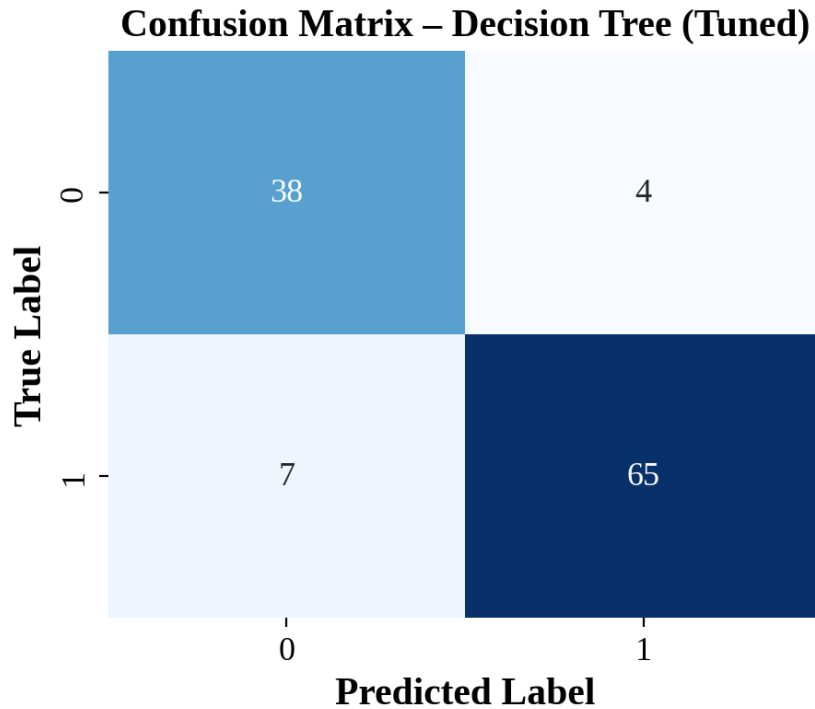


Figure 3: Confusion matrix for the tuned Decision Tree on the held-out test set.

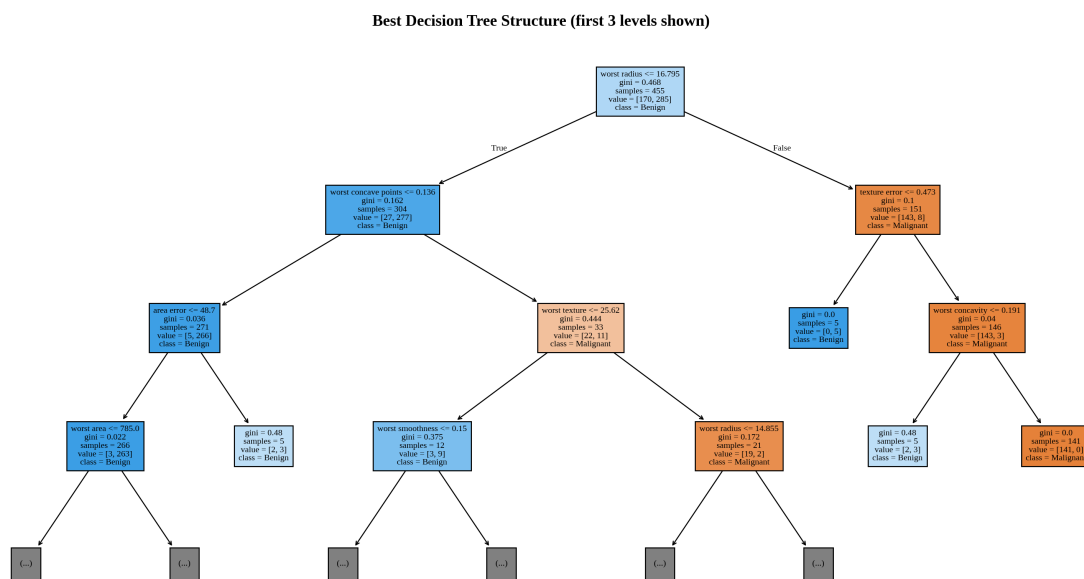


Figure 4: Structure of the tuned Decision Tree (first 3 levels shown for readability).

Inference: The tree's root split is on **worst radius** (≤ 16.795), immediately separating a large, highly-pure benign-leaning branch (277 of 304 benign) from a malignant-leaning branch (143 of 151 malignant). This single split alone achieves strong separation, confirming that tumor size (captured by **worst radius**) is the single most discriminative measurement in this dataset – consistent with the feature importance ranking in Section 10.

9 Overfitting Analysis: Tree Depth vs. Accuracy

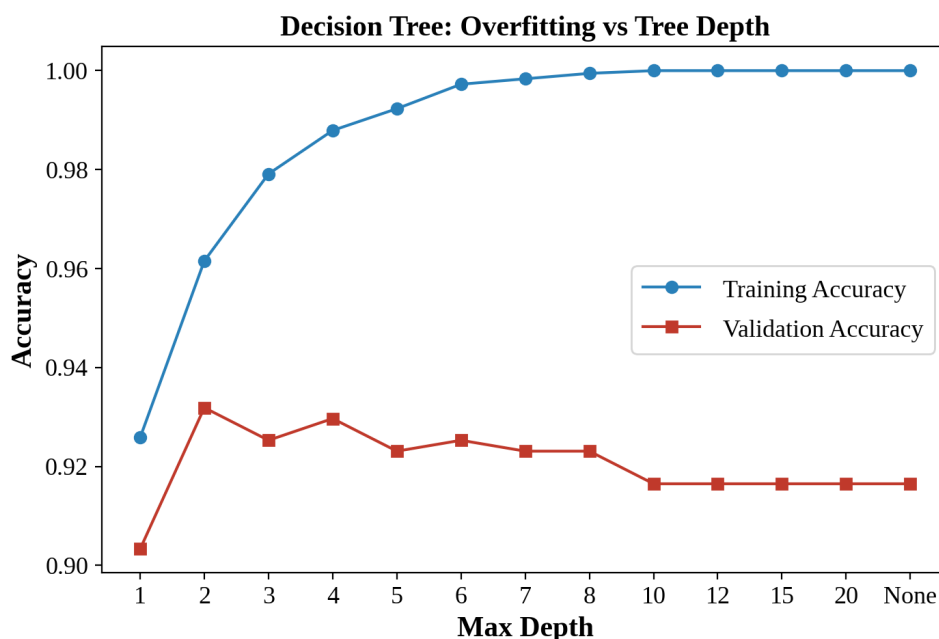


Figure 5: Training vs. 5-fold validation accuracy as a function of maximum tree depth.

Inference: Training accuracy climbs monotonically to a perfect 1.000 by depth 10 and beyond, while validation accuracy peaks early, around depth 2 (0.932), and then *declines* as depth increases further, falling to 0.916 by depth 10 and staying there for all deeper trees. This divergence – training accuracy rising while validation accuracy falls – is the textbook signature of overfitting: beyond a depth of roughly 2–4, the tree begins memorizing noise specific to the training folds rather than learning generalizable decision rules.

10 Random Forest Performance

Metric	Value
Accuracy	0.9561
Precision	0.9561
Recall	0.9561
F1 Score	0.9560
ROC-AUC	0.9924
Training Time (s)	0.1426

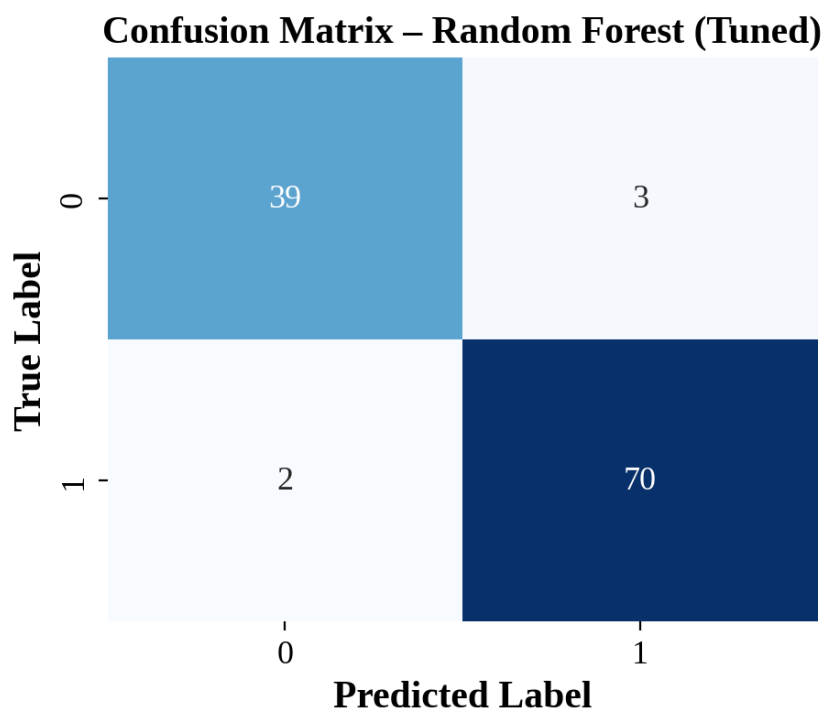


Figure 6: Confusion matrix for the tuned Random Forest on the held-out test set.

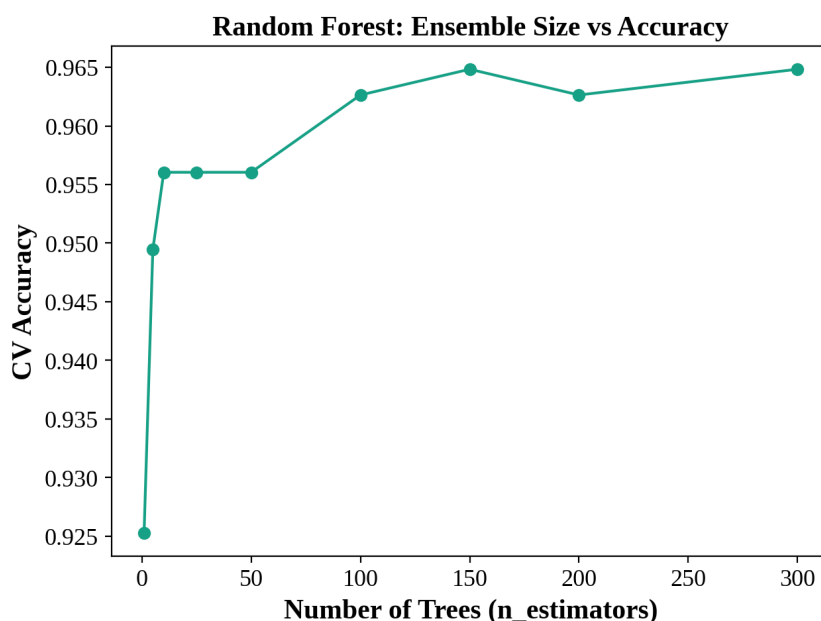


Figure 7: Random Forest: 5-fold CV accuracy as a function of ensemble size ($n_{\text{estimators}}$).

Inference: Accuracy improves quickly from a single tree (0.925, identical to an ordinary Decision Tree) up to about 10–50 trees (0.956), then continues to improve more slowly, plateauing around 150–300 trees (0.965). This is the expected bagging behavior: variance reduction from averaging has diminishing but still-positive returns as more (de-correlated) trees are added, with most of the benefit realized within the first 50–100

trees.

11 5-Fold Cross-Validation Performance Comparison

Table 3: 5-Fold Cross-Validation Accuracy Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.9451	0.9451	0.9121	0.9231	0.9670	0.9385
Random Forest	0.9670	0.9780	0.9341	0.9560	0.9890	0.9648

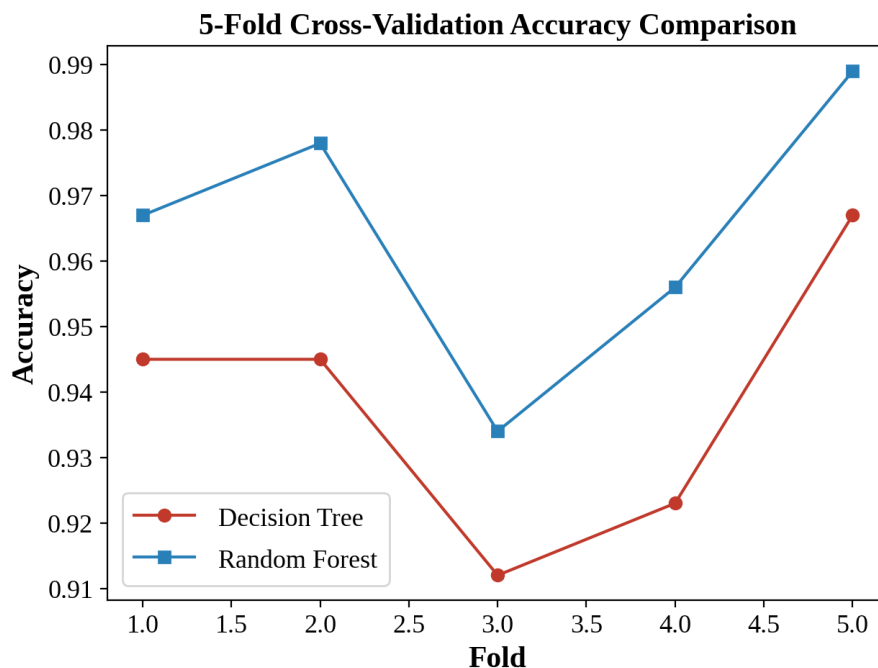


Figure 8: 5-fold cross-validation accuracy: Decision Tree vs. Random Forest.

Inference: Random Forest outperforms the Decision Tree on *every single fold*, not just on average – the two curves never cross. This fold-by-fold consistency is stronger evidence of a genuine improvement than the average alone, since it rules out the possibility that Random Forest’s advantage is driven by a single easy fold.

12 Evaluation Metrics: ROC Curve and Comparative Analysis

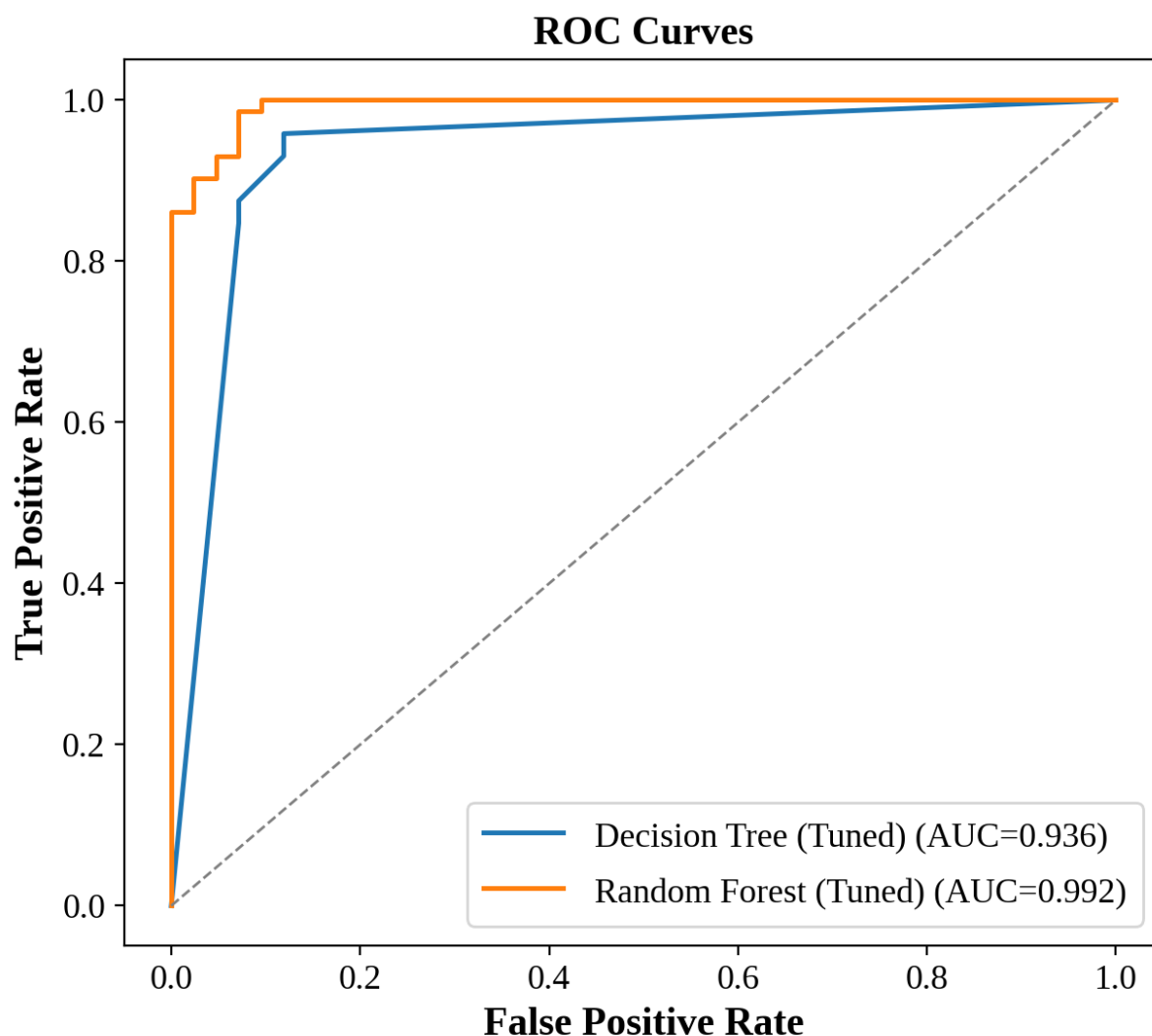


Figure 9: ROC curves for tuned Decision Tree and tuned Random Forest.

Inference: Random Forest's ROC curve dominates the Decision Tree's at every false-positive rate, with a substantially higher AUC (0.9924 vs. 0.9358). For a medical diagnosis task such as this one, where both false negatives (missing a malignant tumor) and false positives (unnecessary biopsy/anxiety for a benign case) carry real costs, this gap represents a practically meaningful improvement in the model's ability to rank malignant cases above benign ones across all possible decision thresholds.

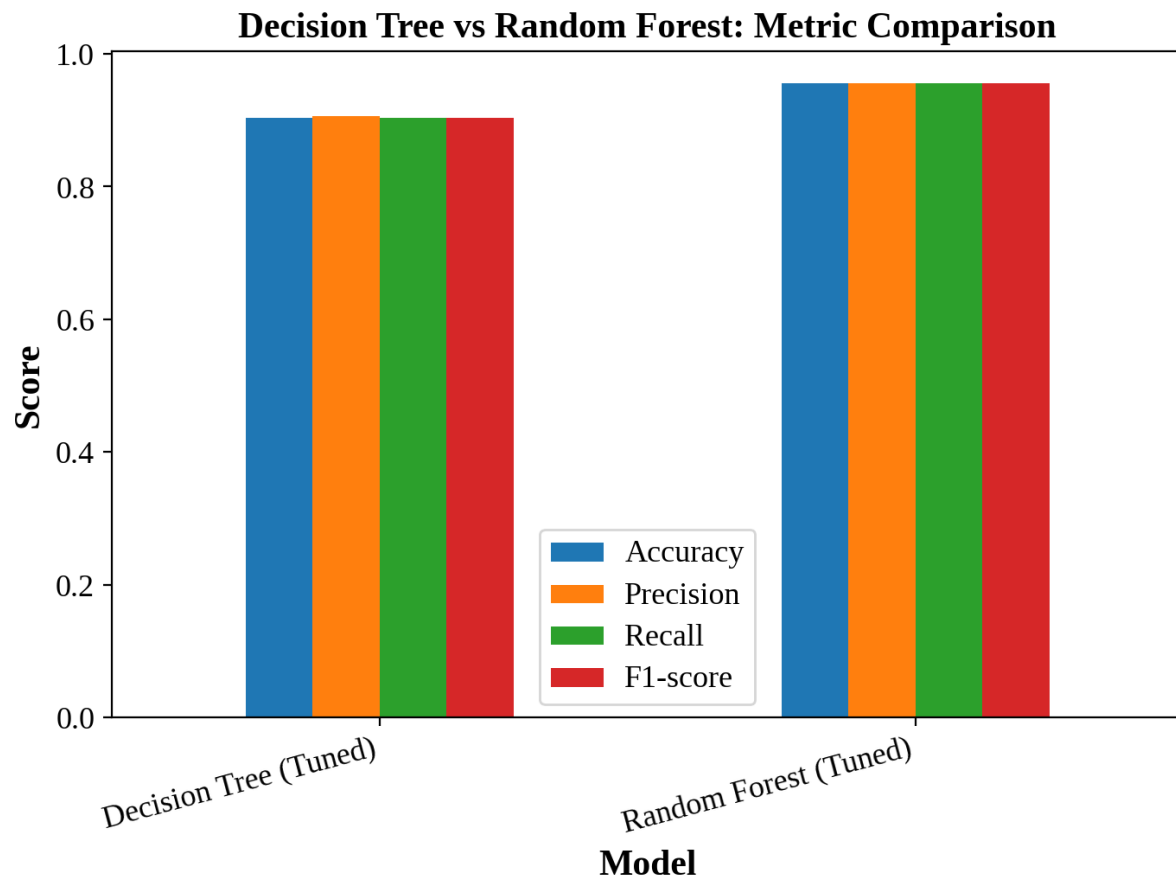


Figure 10: Decision Tree vs. Random Forest: Accuracy, Precision, Recall, and F1 comparison.

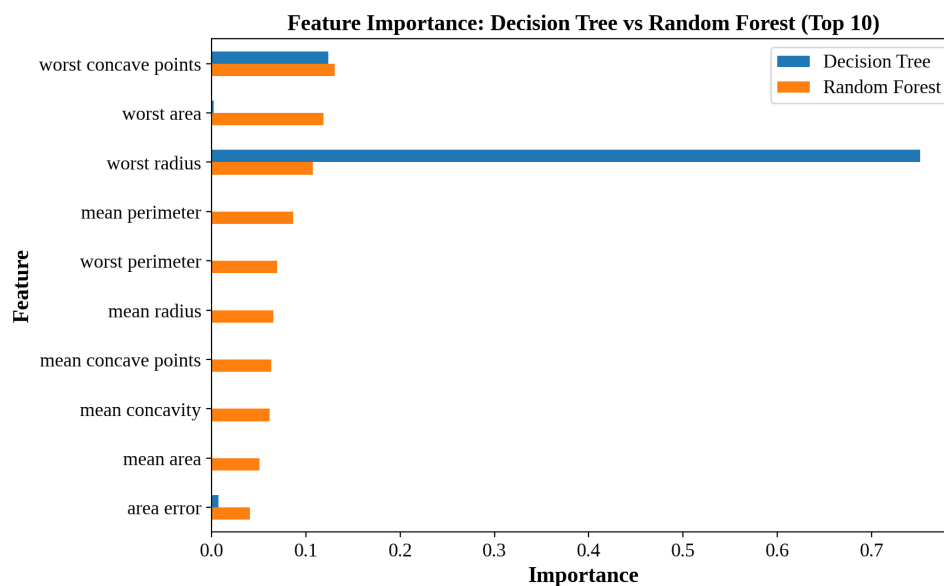


Figure 11: Feature importance comparison: Decision Tree vs. Random Forest (top 10 features).

Inference: The single Decision Tree concentrates almost all of its importance (0.75)

on just one feature, `worst radius`, using only 5 features with non-zero importance in total – a direct consequence of greedy, single-path splitting. Random Forest spreads importance far more evenly across 10+ features (`worst concave points`, `worst area`, `worst radius`, `mean perimeter`, etc., each contributing roughly 5–13%), because each tree in the ensemble is restricted to a random feature subset at every split, forcing the forest as a whole to discover and rely on multiple complementary, correlated but non-identical signals rather than collapsing onto a single dominant feature.

Criterion	Decision Tree	Random Forest
Accuracy (test split)	0.9035	0.9561
Accuracy (5-fold CV)	0.9385	0.9648
ROC-AUC	0.9358	0.9924
Model Complexity	Low	High
Training Time	Low (0.006 s)	Low–Moderate (0.143 s)
Interpretability	High	Low

13 Observations

- **How does tree depth affect overfitting?** As shown in Figure 5, increasing depth beyond about 2–4 causes training accuracy to keep rising toward a perfect 1.0 while validation accuracy peaks and then declines – the tree is fitting noise specific to individual training folds (high variance), a textbook overfitting pattern.
- **Which hyperparameter had the greatest impact on performance?** For the Decision Tree, `max_depth` (in combination with `min_samples_leaf`) had the largest effect, moving CV accuracy by roughly 1.5 percentage points across the tested range and directly controlling the overfitting behavior above; `criterion` (gini vs. entropy) had a much smaller effect (under 1 percentage point). For Random Forest, `n_estimators` had the most visible effect at the low end (Figure 7: +3 points from 1 to 10 trees), while `max_depth`, `max_features`, and `bootstrap` produced smaller, second-order refinements once the ensemble was reasonably large.
- **How does Random Forest improve generalization?** By averaging predictions across many trees, each trained on a different bootstrap sample and restricted to a random feature subset per split, Random Forest decorrelates the individual trees' errors. Errors that are independent across trees partially cancel out under majority voting, reducing the ensemble's overall variance without a corresponding increase in bias – visible here as Random Forest beating the Decision Tree on *every* cross-validation fold (Table 3) despite each individual tree in the forest being no more sophisticated than the single tuned Decision Tree.
- **Did ensemble learning always improve performance? Why or why not?** In this experiment, yes – Random Forest improved every metric tracked (accuracy, precision, recall, F1, ROC-AUC) and every cross-validation fold over the Decision Tree. This is not universally guaranteed: ensemble gains are largest when the base learners are prone to high variance (as deep, unconstrained decision trees are) and when there is enough data to support many bootstrap resamples; on datasets with very few samples, strong label noise, or where a single well-regularized tree

already captures the true decision boundary well, the added complexity and reduced interpretability of a forest may not be worth a marginal (or negligible) accuracy gain.

14 Conclusion

Decision Tree and Random Forest models were implemented and evaluated on the Wisconsin Diagnostic Breast Cancer dataset using 5-fold cross-validation for hyperparameter selection. The tuned Decision Tree (gini, max_depth=5, min_samples_leaf=4) achieved 93.85% CV accuracy and 90.35% test accuracy, while the tuned Random Forest (100 trees, max_depth=10, max_features=log2, bootstrapping enabled) achieved 96.48% CV accuracy and 95.61% test accuracy – a consistent, fold-by-fold improvement across every evaluation metric, most notably raising ROC-AUC from 0.936 to 0.992. These results directly confirm the theoretical expectation that bagging and random feature selection reduce variance and improve generalization relative to a single decision tree, at the cost of reduced interpretability (a single readable tree vs. an opaque 100-tree ensemble) and a roughly 24× increase in training time (still well under a fifth of a second on this dataset size, so practically negligible here). For this diagnostic task, where predictive reliability is paramount, Random Forest is the clearly preferable model; the single Decision Tree remains valuable primarily as an interpretable baseline and as a direct visualization of the dataset’s dominant decision rule (Figure 4).

15 References

1. Scikit-learn Developers, “Decision Trees,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
2. Scikit-learn Developers, “Ensemble methods: Random Forests,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
3. W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu>